

Ms Tanya — Code Frame Spec

The coding contract for the ready-made code frame handed to the juniors — the intelligence is built and tested; the plumbing is marked, owned and listed · v1.0 · Tue 29-Sep-2026

TG Level Pvt Ltd · Project: TG Level Phase 3 AI · Sequence: Research (Architecture v3.2) → **this Spec** → Python (tanya_ai v1.0-frame) → Node Gateway (ported) · Replaces: Ms_Tanya_Demo_Spec_v0.1 (draft, not issued) · For: Tushar Ghone, CODER D, JUNIOR A, JUNIOR B, JUNIOR C

Index

1. What this frame is
2. Rules the code follows
3. Module map — Backbone component, status, owner
4. One turn — the eight steps
5. The label list
6. The decider table
7. Customer Memory — the record, its rules, its Redis keys
8. RAG — knowledge search and the vector index
9. The SEBI guard — three layers
10. Temperature, Lead Brief and Agent Short-Call Card
11. Hand-offs, cases, callbacks and honest times
12. Fixed lines
13. Reliability — lanes, duplicates, retries, dead letter, versions
14. Storage — the 16 orch_ tables and what writes to each
15. AI providers and cost — why no LangChain
16. Content files Tushar replaces
17. Configuration table
18. Tests
19. What is left — plumbing and tuning
20. Open points register
21. Python code — the rule modules, verbatim
 - 21.1 decider.py — The decider table in code
 - 21.2 policy.py — Mode, kill switch, spend ceiling, calling hours
 - 21.3 turn.py — The eight steps of one turn
 - 21.4 graph.py — LangGraph wiring and the plain loop
 - 21.5 memory_model.py — Customer Memory rules
 - 21.6 scoring.py — Lead temperature
 - 21.7 handoff.py — Hand-offs, cases, callbacks, honest times
 - 21.8 guard_input.py — Masking and injection check
 - 21.9 guard_output.py — SEBI net, ₹ check, emoji rule, AI check
 - 21.10 knowledge.py — RAG — knowledge search
22. Change register

1. What this frame is

1. **Your intention (29-Sep-2026):** give the juniors a ready structure with all components already built, so only plumbing (Redis, CRM, servers) and fine tuning remain — instead of 10 days of building.
2. **What is in the frame — written and tested:** the gateway (Python reference), mode policy and kill switch, the understand call, the decider, the prompt builder with persona and golden examples, the knowledge search (RAG), the three-layer SEBI guard, Customer Memory with its rules, temperature, the Lead Brief and Agent Short-Call Card, hand-offs with honest times, cases and callbacks, the 30-question limit and spend ceiling, the provider adapter (Anthropic, OpenAI, Google), LangGraph wiring, Redis Streams workers, the persister to MySQL, the 16 orch_ tables, the morning compliance audit, a developer console and 36 rule tests.
3. **Tested here:** 36 of 36 rule tests pass, including a real Redis (memory, 8 lanes, duplicate drop, version check) and a real MariaDB (all 16 tables created, persister writes); and one full server run — webhook → lane → worker → reply → MySQL.
4. **Not tested here, because it needs your systems:** the real CRM api.php and webhook shapes, real AI keys (the Anthropic endpoint was checked; OpenAI and Google follow their documentation), embeddings, the app's events, the Node port.
5. **Is it 80%?** For the brain — yes, closer to 90%: every decision rule of Architecture v3.2 for the chat pilot is coded. For the whole pilot — the remaining work is real but narrow: CRM plumbing, the loader's field mapping, server setup, the Node port, nudges, content and tuning (Section 19).

2. Rules the code follows

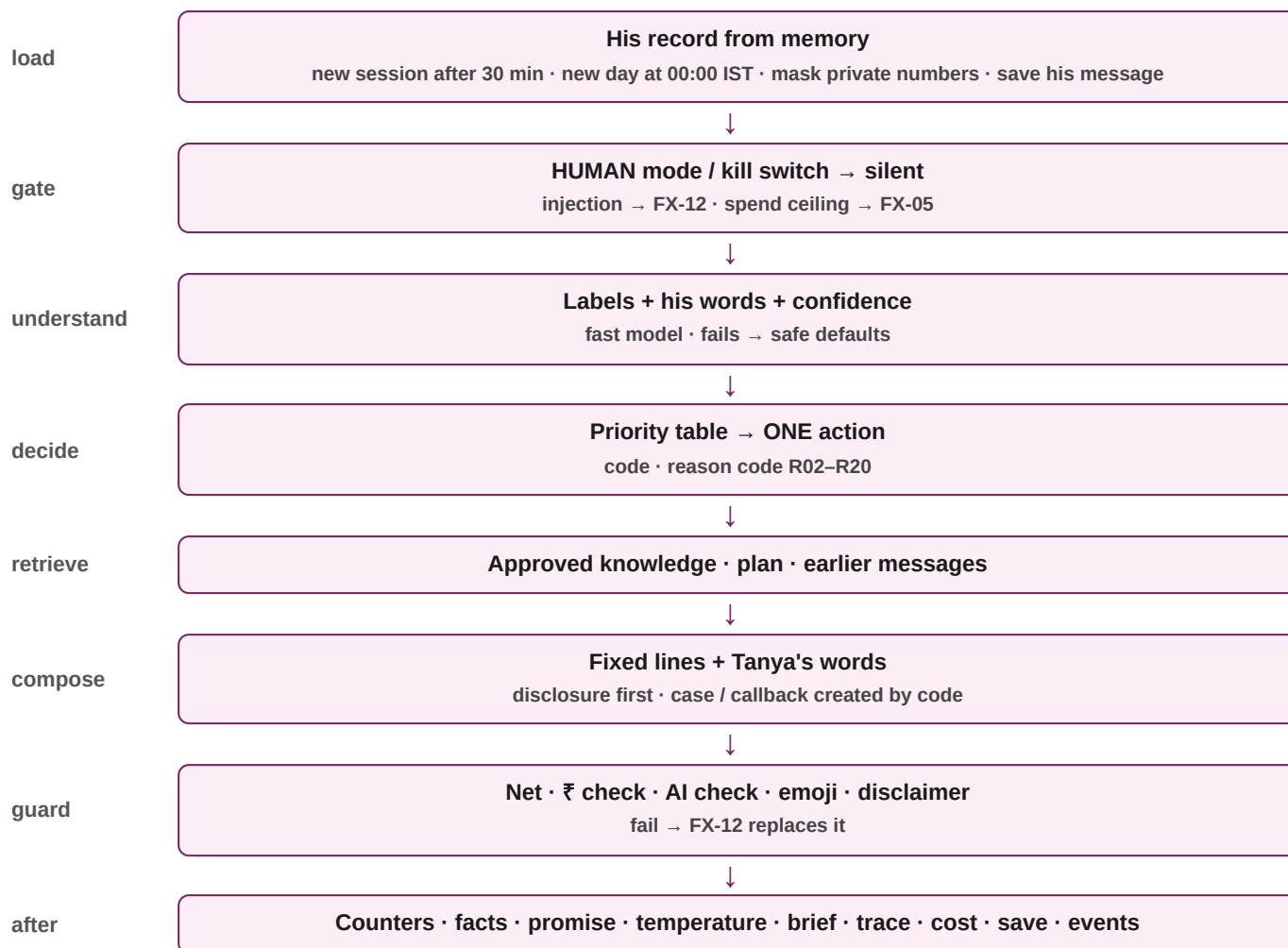
1. **Spec → Python → Node.** A change enters this Spec first, then the Python, then the Node Gateway.
2. **The live chat path never touches MySQL.** Redis is the working memory; the persister writes MySQL behind (Architecture §7.7).
3. **Code decides, the AI writes.** The decider picks one action; the model only writes the words for it.
4. **Fixed lines are never AI-written.** Disclosure, refusals, limits, hand-offs, cases.
5. **Only approved content.** Prices, plans, FAQs and lessons come from the content folder; the AI never invents them.
6. **Nothing reaches him unchecked.** Every AI-written reply passes the net and the AI check.
7. **One persona, one memory, one guard** — built so the voice caller reuses them (Voice Study v1.0).

3. Module map — Backbone component, status, owner

File	Component	Status	Plumbing owner
gateway.py	AI-C01 Gateway (Python reference)	Done; Node port pending	JUNIOR A
policy.py	AI-C02 Mode Policy · AI-C20 Kill Switch · AI-C19 ceiling	Done	—
graph.py, turn.py	AI-C03 Orchestrator (LangGraph + plain loop)	Done	—
understand.py	AI-C03 understand call	Done; tune on real chats	JUNIOR B
decider.py	AI-C03 / AI-C09 decider	Done (v1 + add-ons)	—
prompts.py	AI-C03 / AI-C17 prompt builder	Done; tune with approved examples	JUNIOR B

File	Component	Status	Plumbing owner
knowledge.py	AI-C05 Knowledge / RAG	Done; real content, embedding choice	JUNIOR B
guard_input.py	AI-C23 injection + masking	Done	—
guard_output.py	AI-C14 SEBI guard layers 2–3	Done; 100 trick questions on the chosen model	JUNIOR A
memory_model.py, memory_store.py	AI-C25 Customer Memory (File + Redis)	Done	—
scoring.py, brief.py	AI-C06 / 07 / 09	Done	—
handoff.py	AI-C18 cases, callbacks, honest times	Done; real hand-off target (A8)	JUNIOR A
llm.py, llm_mock.py	AI-C21 Provider Adapter	Done; keys and bake-off	JUNIOR C
streams.py	Redis Streams lanes	Done; deploy + load test	JUNIOR A
workers.py	Turn worker · persister · loader · session closer	Done; loader field mapping	CODER D + JUNIOR C
crm_adapter.py	CRM adapter (Support Board api.php)	Skeleton — TO CONFIRM (A1–A4)	CODER D + JUNIOR C
morning_audit.py	AI-C14 / AI-C22 morning audit	Done (event source); MySQL source + batch	JUNIOR B
db/orch_tables.sql	16 orch_ tables	Done; run on staging, then live	CODER D
static/devconsole.html	Developer console	Done	—

4. One turn — the eight steps



1. **The only branch** is after the gate: go on, jump to compose for a fixed line, or stop silently. Code chooses it.
2. **LangGraph** (Backbone: bounded workflows) wires the eight steps as nodes; `USE_LANGGRAPH=0` runs the same steps in a loop — a test proves both give the same result.
3. **After the turn, the worker** posts each message to the CRM, checking HUMAN mode once more just before posting; a message not delivered is marked undelivered and never remembered as said.

5. The label list

Each yes/no label carries the exact words that prove it and a confidence; it counts only at 0.6 or more.

#	Label	Meaning
1	small_talk	Chat not about trading or the service
2	education_question	A trading or market concept to learn
3	support_question	App, alerts, login, payment, access, install
4	trade_advice_seeking	A view, level, strike, stock, or whether to take an alert
5	distress	Heavy or painful loss, despair, fear — including a calm mention of a big loss
6	grievance	Complaint, refund, "fraud", "cheated"
7	wants_person	Asks for a human

#	Label	Meaning
8	purchase_intent	Wants to buy or pay now (negated → off)
9	interest	Price, plans, what a plan includes
10	timing_objection	Buy later for a reason
11	prefers_call	No time to chat / call me
12	abuse	Abusive words
13	flirting	Romantic or personal questions to Tanya
14	asks_if_ai	Bot or person?
15	not_helpful	Did not help / she repeats
16	refers_to_past	Refers to an earlier chat
—	language · negation · new_facts · mood · complexity	Script; what he does not want; facts in his words (11 allowed fields); mood; fast or detailed model

6. The decider table

The first row that matches wins. Rows R00 and R01 are in the gate (policy.py); R02–R20 in decider.py.

Row	When	Action	Fixed line	Selling
R00	Injection attempt	Safe fallback	FX-12	No
R01	Daily spend ceiling reached	Limit	FX-05	No
R02	distress	PAUSE_SELLING for the session	—	No
R03	grievance	LOG_GRIEVANCE — case number	FX-10	No
R04 / R05	abuse 2nd / 1st time	HAND_OVER_PERSON / BOUNDARY_ABUSE	FX-07·08 / FX-15	No
R06	wants_person, or 3 failed turns	HAND_OVER_PERSON — callback requested	FX-08 day / FX-07 night	No
R07	trade_advice_seeking	REFUSE_AND_TEACH	FX-03	No
R- LIMITED	kill switch "limited" + price / purchase / support	Limited mode	FX-06	No
R08	no consent	ANSWER_ONLY	FX-09 once	No
R09	31st education question today	LIMIT_EDUCATION	FX-04	Offer
R10	purchase_intent, not suppressed	HAND_OVER_PURCHASE — callback	FX-14	Yes
R11	prefers_call	BOOK_CALL — two slots	FX-13	—
R12	asks_if_ai	Honest answer	FX-11	No
R13	interest, not suppressed	ANSWER_PRICE — pricing file only	—	Yes
R14 / R15	flirting 2nd / 1st time	BOUNDARY_FIRM / BOUNDARY_LIGHT	FX-16 / —	No
R16	education_question	ANSWER_EDUCATION (+ one add-on)	FX-17 once	Add-on
R17	support_question	ANSWER_SUPPORT — no approved answer → case	FX-19	No
R18 / R19	small talk after 2 turns / small talk	GUIDE_NEXT_STEP / ANSWER_SMALL_TALK	—	No
R20	anything else	ANSWER (+ one add-on)	—	Add-on

1. **Add-ons (one at most, never when suppressed):** ASK_MISSING_FACT — first unknown of experience, segment, main_pain, time_available, goal; at most 2 per session. SHOW_VALUE_CARD — from his 3rd message, card matching his pain in his words, not shown before.
2. **Always:** FX-01 disclosure before his first reply ever; FX-02 chat-first line once after the first useful answer.
3. **Suppression** = distress this session, an open grievance, or no consent — blocks R10, R13, add-ons and Hot.

7. Customer Memory — the record, its rules, its Redis keys

1. **The record per user:** profile (name, language, consent, trial start) · facts (value, his words, source, message number, date-time, stated, confidence, history) · conflicts · journey (disclosed, chat-first given, last promise, cards shown, greeted at) · session (id, profile questions, small talk, failed, abuse, flirt, selling paused, disclaimer shown) · daily counters · signals with time · cases · callbacks · mode · last 300 messages · version.
2. **Which fact wins:** his latest words in conversation; an agent-note or CRM value that differs is never changed and becomes a conflict for the agent (shown only after shadow mode).
3. **Never a source:** her own replies and her own brief.
4. **No consent:** no facts, no signals, no brief — answer only.
5. **Cold start:** a user not yet loaded is answered only (no profiling, no selling) while the loader fills his record from the CRM.
6. **Safe writes:** every save checks the version it read; an older writer is refused and redoes its turn (Redis WATCH / MULTI).

Redis key	Holds
tanya:rec:{user_id}	The record (JSON) with its version
tanya:in:0 ... tanya:in:7	Intake lanes — one customer always in the same lane
tanya:seen:{event_id}	Duplicate guard, 24 hours
tanya:human:{conversation_id}	HUMAN flag set at once by the webhook receiver; expires by itself
tanya:persist	Events for the MySQL persister
tanya 🇮🇳 {date} · tanya:caseseq:{date}	Spend today in ₹ · case number counter
tanya:killswitch	off · limited · stopped
tanya:loadq · tanya:dead · tanya:errors	Loader queue · dead letters · errors

8. RAG — knowledge search and the vector index

1. **Source:** approved files in content/kb (one per topic: header, question forms, answer, never-say).
2. **Pieces:** each answer becomes one or more chunks (split at blank lines above 900 characters).
3. **Search:** keyword (BM25, with a Hinglish-to-English word list) always; plus meaning search with vectors when EMBEDDING_PROVIDER is set — the two scores averaged ("hybrid").
4. **Vector index:** for the pilot, held in memory with vectors cached in data/kb_vectors.json per content version; the same chunks go to orch_kb_chunks; the Backbone's vector DB (Qdrant or MongoDB Vector Search, decision open) replaces the in-memory index later without changing callers.
5. **Placeholder answers are never used:** a support question whose best answer is a placeholder becomes a case (FX-19).
6. **Customer memory is not in the vector index** — it is looked up by user ID.

9. The SEBI guard — three layers

1. **Layer 1 — route:** trade-advice questions → REFUSE_AND_TEACH: fixed refusal + a lesson that always starts from Logic · Risk · Exit.
2. **Layer 2 — AI check:** a separate fast-model call fails the reply for a direction view; a level, strike or named instrument to trade; a return, accuracy or recovery promise; a price or benefit not in the approved text; an

invented human life; "tum"; superlatives or fake urgency. A failed check call counts as a fail.

3. **Layer 3 — net (code)**: promise patterns allowed only with a denial in the same sentence; hard patterns never (strike, buy/sell an instrument, target, stop-loss or entry level, direction certainty, superlatives, fake urgency); every ₹ figure must be a pricing-file price, a figure he said, or one in the knowledge used; emoji rule; FX-17 disclaimer once per session.
4. **Any fail** → **FX-12** replaces the reply, the failed turn is counted, and the blocked text is kept in the trace.

10. Temperature, Lead Brief and Agent Short-Call Card

1. **Hot** = purchase intent (48 h) + readiness (experience and segment known, no timing objection, no refused call) + no suppression. **Warm** = interest (72 h) + experience + pain known. **Nurture** = 3+ messages, no intent. Else **Not ready**.
2. **Lead Brief** (code, no AI cost): temperature with evidence, trial day, consent, facts in his words with date-time, plan by the profile table, last promise, cases, callbacks, channel preference; conflicts only after shadow mode. Written to the CRM at every hand-over and at session end.
3. **Agent Short-Call Card** (Architecture §13.18): purpose, 5–7 minute time box, who he is, do-not-ask list, his words, covered in chat, plan to mention, compliance lines, close.

11. Hand-offs, cases, callbacks and honest times

1. **Inside calling hours**: "a team member will join shortly" (FX-08). **Outside**: the next opening time in words — "kal subah 10 baje" (FX-07).
2. **Callbacks**: person · purchase · call preference; states requested → booked → completed; one open callback per kind.
3. **Call preference**: two whole-hour slots inside calling hours, at least 1 hour ahead and 3 hours apart (FX-13).
4. **Cases**: grievance and support, numbered TG-YYYYMMDD-NNN.

12. Fixed lines

In content/fixed_lines.json, in English, Hinglish and Hindi — all DRAFT for your and compliance approval (input D2).

ID	Moment	ID	Moment
FX-01	AI disclosure — first message ever	FX-11	"Are you a real person?"
FX-02	Chat-first line	FX-12	Safe fallback
FX-03	Refusal of a trade question	FX-13	Call preference — two slots
FX-04	30 education questions reached	FX-14	Purchase — senior for formalities
FX-05	Spend ceiling reached	FX-15	Abuse — calm line
FX-06	Limited mode	FX-16	Repeated flirting — firm line
FX-07	Hand-off outside calling hours	FX-17	Disclaimer (TO CONFIRM — D1)
FX-08	Hand-off inside calling hours	FX-18	SEBI registration line (TO CONFIRM — D1)
FX-09	No consent in the app	FX-19	Support question passed to the team with a case
FX-10	Grievance with case number		

13. Reliability — lanes, duplicates, retries, dead letter, versions

- 1. **Order per customer:** 8 lanes chosen by his ID; one owner per lane.
- 2. **Duplicates:** the same webhook event ID is dropped for 24 hours.
- 3. **Retries:** a job not confirmed in 60 s is picked up again; after 5 deliveries it goes to tanya:dead (and orch_dead_letters).
- 4. **Versions:** an older writer can never overwrite a newer record; the turn is redone.
- 5. **HUMAN at once:** the webhook receiver sets the HUMAN flag before queueing a staff message; the worker checks it again before posting.
- 6. **Left for JUNIOR A (S5.1):** the catch-up poller (every 60 s via api.php) and the nightly Redis ⇌ MySQL audit.

14. Storage — the 16 orch_ tables and what writes to each

Event	Table
his message	orch_inbox
her message (line ID, action, delivered)	orch_replies
fact	orch_lead_facts
signal	orch_lead_signals
case	orch_lead_events + orch_alerts
callback	orch_callbacks
trace (and each AI call)	orch_reply_trace + orch_ai_usage
state (only if newer version)	orch_lead_state
brief · session note	orch_ai_notes
mode change	orch_conversation_state
session start / end, value card	orch_lead_events
morning audit flag	orch_compliance_audits
knowledge chunk	orch_kb_chunks
dead letter	orch_dead_letters
pilot group	orch_pilot_groups

15. AI providers and cost — why no LangChain

- 1. **One door:** every AI call goes through `LLM.call(purpose, tier, ...)` — understand, reply, check, note, audit.
- 2. **Providers:** Anthropic (Haiku 4.5 / Sonnet 5.5), OpenAI (GPT-6 Luna / Sol), Google (Gemini 3.8 Flash / 3.1 Pro) — set in .env; a fallback provider is tried if the first fails; the morning audit can use a different provider.
- 3. **Cost:** tokens × price table × ₹96 per US\$, per call, in the trace and the ledger.
- 4. **Why not LangChain:** direct HTTPS keeps prompts, time limits, cost and fallback visible and testable; the Backbone does not require it.

16. Content files Tushar replaces

File	Input
content/pricing.csv	C1
content/profile_to_plan.csv	C2
content/kb/*.md	C3, C4
content/value_cards.json	C5
content/fixed_lines.json	D2
content/sebi_texts.json	D1
content/golden_examples.json	Personality Guide approval
content/guard_lists.json	D3
content/test_users.json	A5, F2 (test only)

17. Configuration table

Owner: **Tushar** = your decision · **Claude** = my proposal · **Open** = waiting for the decision named.

#	Parameter	Value	Owner
1	trial_length_days	3	Open — G1 (3 or 5)
2	education_questions_per_day	30	Tushar, 25-Sep
3	failed_turns_for_handoff	3	Tushar, 25-Sep
4	profile_questions_per_session	2	Claude — Architecture §7.2
5	session_gap_minutes	30	Claude — Architecture §7.6
6	small_talk_turns_before_bridge	2	Claude — Personality Guide §6.3
7	value_card_min_user_messages	3	Claude
8	label_confidence_min	0.6	Claude
9	calling_window	10:00–19:00	Open — Architecture §21.2.1
10	interest_expiry_hours / intent_expiry_hours	72 / 48	Claude — Architecture §8
11	daily_ai_spend_ceiling_inr	200 (dev)	Open — E2
12	spend_alert_percent	80	Claude — Architecture §17
13	usd_to_inr	96	Claude — 25-Sep-2026
14	temperatures understand / reply / check	0.0 / 0.4 / 0.0	Claude — bake-off tunes
15	max_lines chat / teaching	3 / 6	Claude — Personality Guide §3
16	disclaimer_once_per_session	yes	Claude
17	knowledge_top_k · golden_examples_per_prompt	3 · 6	Claude
18	abuse_count_for_handoff	2	Claude — Personality Guide §5.3
19	contradictions_visible_to_agents	no (shadow)	Claude — Architecture §9.4.7
20	history_messages_in_prompt	10	Claude — Architecture §7.6
21	timeouts understand / reply / check	20 / 40 / 20 s (dev)	Claude — production per Architecture §15
22	human_mode_release_hours	12	Claude — change if you prefer

#	Parameter	Value	Owner
23	greeting_min_gap_hours	3	Claude
24	stream_partitions · max_deliveries · reclaim_idle_seconds	8 · 5 · 60	Claude — v4 design
25	models, prices, embedding models	Section 15	Tushar — keys; Claude — choice

18. Tests

1. **Rule tests — `run_tests.bat`, no key:** masking; injection; decider rows (distress over purchase, refusal, grievance, purchase, open grievance blocks selling, no consent, 31st question, 3 failed turns, known fact never asked, 2 questions per session, flirting, abuse); honest night time; call slots; net (levels, denial allowed, promise, ₹ figures); emoji rule; conflict recorded; suppression never Hot; full turns (disclosure, loss pauses selling, refusal, HUMAN silence, kill switch, ceiling, support case, night hand-off); LangGraph equals plain loop; knowledge finds the stop-loss lesson. **With Redis:** memory, stale write refused, lanes, duplicate drop. **With MySQL:** persister writes replies, facts, callbacks.
2. **Result on 29-Sep-2026:** 36 passed, 0 failed.
3. **Live tests — `tests/run_golden.py`, needs a key:** voice tests from the Personality Guide and the trick questions (40 drafted, 100 required — D3); report in `data/golden_report.md`; zero failures is the release gate.

19. What is left — plumbing and tuning

1. **CODER D:** S0.1 CRM proofs (fills every TO CONFIRM in `crm_adapter.py`), S0.2 tables on staging, loader field mapping with JUNIOR C.
2. **JUNIOR A:** Node Gateway port, Redis and workers on the server, catch-up poller, nightly Redis ⇌ MySQL audit, real hand-off target, 100 trick questions.
3. **JUNIOR B:** bake-off runs, tuning on real chats, golden examples, morning audit on MySQL with the batch API.
4. **JUNIOR C:** CRM adapter and loader, app-open event, live status, alerts to Telegram or email, load test, pilot readiness.
5. **Not in the frame:** trial-day nudges (needs push API and G1), objection library and commitment ladder (decider v2), catch-up poller, nightly audit.
6. **Estimate:** about 5–6 working days in parallel to a closed pilot — only if the P0 inputs arrive on Day 0 (`docs/JUNIOR_TASKS.md`).

20. Open points register

Every point is either an input still to come or a decision, each with its effect today.

#	Open point	Effect in the frame today	Closed by
1	CRM function names and payloads	SupportBoardAdapter marked TO CONFIRM; console adapter used	A1–A4 (CODER D)
2	CRM user fields and agent-note format	Loader maps assumed names	A5, A4
3	App events and consent field	Only <code>app_open</code> handled; consent from the record	B2, B3
4	Prices, plan inclusions, plan table	Placeholders; inclusions never described	C1, C2
5	FAQ answers and lessons	Lessons drafted; FAQs placeholder → case	C3, C4

#	Open point	Effect in the frame today	Closed by
6	SEBI registration and disclaimer text	FX-17 / FX-18 placeholders	D1
7	Trial length 3 or 5 days	3 used	G1
8	Calling window	10:00–19:00 used	Architecture §21.2.1
9	Spend ceiling	₹200 a day (dev)	E2
10	Model choice	Any of three by .env	Bake-off (S2.7)
11	Vector DB	In-memory index	Backbone decision
12	Nudges, objection library, catch-up poller, nightly audit	Not built	Build Flow S5.1–S5.3

21. Python code — the rule modules, verbatim

1. **Why it is here:** your rule — the Spec carries the Python itself, so nobody guesses how a rule was coded. The Node Gateway and any later change are derived from this code.
2. **How to read it:** every module opens with a plain-English note (the text between the triple quotes), and every function carries one line saying what it does. Lines starting with # are notes, not code.
3. **Printed from:** tanya_ai v1.0-frame, Tue 29-Sep-2026 — the same files as in the zip. The other modules (gateway, workers, streams, provider adapter, CRM adapter, prompts, morning audit) are plumbing or text and live in the zip.

21.1 decider.py — The decider table in code

```
"""AI-C03 / AI-C09 — the Next-Action decider (Architecture v3.2 §6.2–6.3; Code Frame Spec §5).

Plain English: this is CODE, not AI. It reads the labels from the understand
call and his memory record, and picks ONE action for this turn by walking
down the priority table — the first row that matches wins. Every decision
carries a reason code (R01..R20) that goes into the trace.

The AI later writes the words for that action; it never chooses the action.
"""

from dataclasses import dataclass, field

from . import memory_model as mm
from .settings import S

# Actions whose whole reply is a fixed line (no AI writing)
FIXED_ONLY = {"LIMIT_SPEND", "BOUNDARY_ABUSE", "HAND_OVER_PERSON", "LIMIT_EDUCATION",
              "HAND_OVER_PURCHASE", "BOOK_CALL", "ASKS_IF_AI", "BOUNDARY_FIRM", "LIMITED_MODE"}

# Actions that count toward the 30 education questions
COUNTS_AS_EDUCATION = {"ANSWER_EDUCATION"}

# Actions where emoji are not allowed (Personality Guide §3.5)
NO_EMOJI = {"PAUSE_SELLING", "LOG_GRIEVANCE", "BOUNDARY_ABUSE", "REFUSE_AND_TEACH", "HAND_OVER_PERSON",
            "HAND_OVER_PURCHASE", "ANSWER_PRICE", "LIMIT_SPEND", "LIMIT_EDUCATION", "BOUNDARY_FIRM"}

@dataclass
class Decision:
    action: str
    reason: str
    fixed_line: str = "" # FX-xx shown before or instead of AI words
    selling_allowed: bool = False
    addon: str = "" # ASK_MISSING_FACT | SHOW_VALUE_CARD | ""
    addon_detail: dict = field(default_factory=dict)
    tier: str = "fast" # fast | detailed model
    notes: list = field(default_factory=list)

def _on(labels, name) -> bool:
    return labels["labels"].get(name, {}).get("on", False)

def decide(labels: dict, rec: dict, now, limited: bool = False) -> Decision:
    """Walk the priority table (Spec §5). labels = normalised understand output."""
    sess = rec["session"]
    suppressed, why = mm.suppression(rec)
    consent = rec["profile"].get("consent", False)
    tier = "detailed" if labels.get("complexity") == "detailed" else "fast"

    # Row 2 — distress: support first, selling paused for the session
    if _on(labels, "distress"):
        return Decision("PAUSE_SELLING", "R02", notes=["selling paused for this session"])
    # Row 3 — grievance: case logged, handled first
    if _on(labels, "grievance"):
        return Decision("LOG_GRIEVANCE", "R03", fixed_line="FX-10")
    # Rows 4–5 — abuse (count already includes this message)
    if _on(labels, "abuse"):
        if sess.get("abuse", 0) >= S.get("abuse_count_for_handoff", 2):
            return Decision("HAND_OVER_PERSON", "R04")
        return Decision("BOUNDARY_ABUSE", "R05", fixed_line="FX-15")
    # Row 6 — asks for a person, or 3 failed turns in this session
    if _on(labels, "wants_person") or sess.get("failed", 0) >= S.get("failed_turns_for_handoff", 3):
        reason = "R06" if _on(labels, "wants_person") else "R06-FAILED"
```

```

        return Decision("HAND_OVER_PERSON", reason)
# Row 7 – trade advice: refuse and teach (SEBI guard layer 1)
if _on(labels, "trade_advice_seeking"):
    return Decision("REFUSE_AND_TEACH", "R07", fixed_line="FX-03", tier="fast")
# Kill switch 'limited': general and education only (Architecture §7.5)
if limited and (_on(labels, "purchase_intent") or _on(labels, "interest") or _on(labels,
"support_question")):
    return Decision("LIMITED_MODE", "R-LIMITED", fixed_line="FX-06")
# Row 8 – no consent: answer only
if not consent:
    d = Decision("ANSWER_ONLY", "R08", tier=tier)
    d.fixed_line = "FX-09" if not rec["journey"].get("consent_line_given") else ""
    return d
# Row 9 – education limit
if _on(labels, "education_question") and \
    rec["counters"].get("education_used", 0) >= S.get("education_questions_per_day", 30):
    return Decision("LIMIT_EDUCATION", "R09", fixed_line="FX-04")
# Row 10 – purchase intent, no suppression
if _on(labels, "purchase_intent") and not suppressed:
    return Decision("HAND_OVER_PURCHASE", "R10", fixed_line="FX-14", selling_allowed=True)
# Row 11 – his preference is a call
if _on(labels, "prefers_call"):
    return Decision("BOOK_CALL", "R11", fixed_line="FX-13")
# Row 12 – is she an AI?
if _on(labels, "asks_if_ai"):
    return Decision("ASKS_IF_AI", "R12", fixed_line="FX-11")
# Row 13 – price / plan interest, no suppression
if _on(labels, "interest") and not suppressed:
    return Decision("ANSWER_PRICE", "R13", selling_allowed=True)
# Rows 14-15 – flirting (count includes this message)
if _on(labels, "flirting"):
    if sess.get("flirt", 0) >= 2:
        return Decision("BOUNDARY_FIRM", "R14", fixed_line="FX-16")
        return Decision("BOUNDARY_LIGHT", "R15")
# Row 16 – education question (+ one add-on)
if _on(labels, "education_question"):
    d = Decision("ANSWER_EDUCATION", "R16", tier=tier)
    _addon(d, rec, suppressed)
    return d
# Row 17 – support question
if _on(labels, "support_question"):
    return Decision("ANSWER_SUPPORT", "R17")
# Rows 18-19 – small talk, bridge after N turns (count includes this message)
if _on(labels, "small_talk"):
    if sess.get("small_talk", 0) > S.get("small_talk_turns_before_bridge", 2):
        return Decision("GUIDE_NEXT_STEP", "R18")
        return Decision("ANSWER_SMALL_TALK", "R19")
# Row 20 – anything else
d = Decision("ANSWER", "R20", tier=tier)
_addon(d, rec, suppressed)
return d

def _addon(d: Decision, rec: dict, suppressed: bool):
    """At most one add-on, only after his own question is answered, never when suppressed."""
    if suppressed:
        return
    sess = rec["session"]
    missing = mm.missing_profile_fields(rec)
    if missing and sess.get("profile_q", 0) < S.get("profile_questions_per_session", 2):
        d.addon, d.addon_detail = "ASK_MISSING_FACT", {"field": missing[0]}
        return
    from .content_pack import PACK
    if rec["counters"].get("user_msgs_total", 0) >= S.get("value_card_min_user_messages", 3):
        card = PACK.card_for(mm.pain_text(rec), rec["journey"].get("cards_shown", []))
        if card:
            d.addon, d.addon_detail = "SHOW_VALUE_CARD", {"card": card["id"]}
            d.selling_allowed = True

```

21.2 policy.py — Mode, kill switch, spend ceiling, calling hours

```

"""AI-C02 Mode Policy Engine + AI-C20 Kill Switch / Safe Mode + AI-C19 spend ceiling.

```

Plain English: before Tanya thinks about a message, these rules decide whether she may answer at all, and in what mode:

- HUMAN mode: a staff member is handling the chat → Tanya stays silent.
- Kill switch: 'stopped' → silent; 'limited' → general and education answers only.
- Spend ceiling: at 100% of the day's AI budget → fixed line FX-05.
- Consent: no DPDP agreement in the app → answer only, nothing remembered or sold.

```

from .settings import S
from .timeutil import iso, parse, hm

GATE_GO = "go"
GATE_SILENT = "silent"      # no reply at all (HUMAN mode, kill switch stopped)
GATE_FIXED = "fixed"       # a fixed line only, no AI call

def human_takeover(rec, now, by="staff"):
    """A staff reply arrived → HUMAN mode at once (v4 rule)."""
    rec["mode"] = {"state": "HUMAN", "since": iso(now), "by": by}

def release_to_bot(rec, now, by="staff"):
    rec["mode"] = {"state": "BOT", "since": iso(now), "by": by}

def mode_is_human(rec, now) -> bool:
    """HUMAN mode ends by itself after human_mode_release_hours without staff activity."""
    m = rec.get("mode", {})
    if m.get("state") != "HUMAN":
        return False
    hours = S.get("human_mode_release_hours", 12)
    if (now - parse(m["since"])).total_seconds() > hours * 3600:
        release_to_bot(rec, now, by="timeout")
        return False
    return True

def gate(rec, store, now, injection_flag: bool):
    """Returns (decision, fixed_line_id or None, reason_code)."""
    ks = store.killswitch()
    if ks == "stopped":
        return GATE_SILENT, None, "KILL_STOPPED"
    if store.human_flag(rec["conversation_id"], now) and rec["mode"].get("state") != "HUMAN":
        human_takeover(rec, now, by="webhook flag")
    if mode_is_human(rec, now) or store.human_flag(rec["conversation_id"], now):
        return GATE_SILENT, None, "HUMAN_MODE"
    if injection_flag:
        return GATE_FIXED, "FX-12", "R00"
    ceiling = S.get("daily_ai_spend_ceiling_inr", 200)
    if store.ledger_get(now) >= ceiling:
        return GATE_FIXED, "FX-05", "R01"
    return GATE_GO, None, "OK"

def spend_alert(store, now) -> bool:
    """True once spend passes the alert share of the ceiling (80%)."""
    return store.ledger_get(now) >= S.get("daily_ai_spend_ceiling_inr", 200) * S.get("spend_alert_percent",

80) / 100

def in_calling_hours(now) -> bool:
    sh, sm = hm(S.get("calling_window_start", "10:00"))
    eh, em = hm(S.get("calling_window_end", "19:00"))
    t = now.hour * 60 + now.minute
    return sh * 60 + sm <= t < eh * 60 + em

```

21.3 turn.py — The eight steps of one turn

```

"""AI-C03 Sales Chat Orchestrator – one turn of Ms Tanya, as a chain of steps ('nodes').

Plain English – the steps, in order (Architecture v3.2 §5.2):
load      → his record from memory (Redis in production), new session / new day, mask private numbers
gate      → HUMAN mode? kill switch? injection? spend ceiling? (may stop here or give a fixed line)
understand → labels with evidence (AI, fast model)
decide    → ONE action by the priority table (code)
retrieve  → approved knowledge / plan / earlier messages
compose   → fixed lines + Tanya's written reply (AI)
guard     → SEBI net + AI check + emoji rule + disclaimer
after     → counters, facts, promise, cases, temperature, Lead Brief, trace, cost, save, events
Every node takes the 'state' dict and returns it. graph.py wires them (LangGraph or plain loop).
"""
import re

from . import memory_model as mm
from . import policy
from .brief import lead_brief
from .content_pack import PACK
from .decider import Decision, FIXED_ONLY, COUNTS_AS_EDUCATION, NO_EMOJI, decide

```

```

from .guard_input import injection, mask
from .guard_output import ai_check, amounts, approved_text_for, emoji_rule, net
from .handoff import open_case, request_callback, two_slots, when_phrase
from .knowledge import is_placeholder
from .prompts import build
from .scoring import temperature
from .settings import S
from .timeutil import iso, parse, stamp
from .understand import defaults, understand

USEFUL_ANSWERS = {"ANSWER_EDUCATION", "ANSWER", "ANSWER_SUPPORT", "ANSWER_PRICE", "ANSWER_ONLY"}

def _ev(st, etype, **data):
    st["events"].append({"type": etype, "user_id": st["user_id"], "at": iso(st["now"]), **data})

def _usage(st, res):
    st["llm_calls"].append({"purpose": res.purpose, "provider": res.provider, "model": res.model,
                            "in": res.input_tokens, "out": res.output_tokens, "ms": res.ms,
                            "cost_inr": res.cost_inr, "ok": res.ok, "error": res.error})

# ----- load
def n_load(st):
    now, store, uid = st["now"], st["store"], st["user_id"]
    rec = store.get(uid)
    st["cold_start"] = False
    if rec is None:
        seed = st["seed_fn"](uid)
        if seed is None:
            # not in memory and not loaded yet (Architecture §7.7.5)
            seed, st["cold_start"] = {"consent": False}, True
        rec = mm.new_record(uid, seed, now)
    st["rec"] = rec
    st["new_session"] = mm.ensure_session(rec, now)
    mm.ensure_day(rec, now)
    if st["new_session"]:
        _ev(st, "session_start", session=rec["session"]["id"])
    if st["kind"] == "message":
        masked, kinds = mask(st["text"])
        st["masked"], st["masked_kinds"], st["injection"] = masked, kinds, injection(st["text"])
        st["msg_no"] = mm.add_message(rec, "user", masked, now, meta={"masked": kinds, "event_id":
st.get("event_id")})
        rec["session"]["user_msgs"] += 1
        rec["counters"]["user_msgs_total"] += 1
        _ev(st, "message", role="user", n=st["msg_no"], text=masked, event_id=st.get("event_id"))
    else:
        st["masked"], st["masked_kinds"], st["injection"], st["msg_no"] = "", [], False, None
    return st

# ----- gate
def n_gate(st):
    rec, now = st["rec"], st["now"]
    g, line, reason = policy.gate(rec, st["store"], now, st["injection"])
    st["limited"] = st["store"].killswitch() == "limited"
    if st["kind"] == "app_open" and g == policy.GATE_G0:
        last_greet = rec["journey"].get("greeted_at")
        if last_greet and (now - parse(last_greet)).total_seconds() < S.get("greeting_min_gap_hours", 3) *
3600:
            g, reason = policy.GATE_SILENT, "GREETED_RECENTLY"
    st["gate"], st["gate_line"], st["gate_reason"] = g, line, reason
    if g == policy.GATE_FIXED:
        st["decision"] = Decision("FIXED_GATE", reason, fixed_line=line)
        st["labels"] = defaults(st["masked"])
    return st

def route_after_gate(st):
    return {"go": "understand", "fixed": "compose", "silent": "after"}[st["gate"]]

# ----- understand
def n_understand(st):
    rec, now = st["rec"], st["now"]
    if st["kind"] != "message":
        st["labels"] = defaults("")
        st["labels"]["language"] = rec["profile"].get("language", "hinglish")
        st["labels"]["failed"] = False
    return st
    history = mm.history_for_prompt(rec)[-1]
    labels, res = understand(st["llm"], st["masked"], history)

```

```

_usage(st, res)
st["labels"] = labels
L = labels["labels"]
sess = rec["session"]
rec["profile"]["language"] = labels["language"]
# --- session counters (this message included) ---
if L["small_talk"]["on"]:
    sess["small_talk"] += 1
if L["abuse"]["on"]:
    sess["abuse"] += 1
if L["flirting"]["on"]:
    sess["flirt"] += 1
if L["not_helpful"]["on"] or labels.get("failed"):
    sess["failed"] += 1
# --- facts and signals: only with consent (DPDP agreement in the app) ---
if rec["profile"].get("consent"):
    for f in labels["new_facts"]:
        mm.set_fact(rec, f["field"], f["value"], f["his_words"], "chat", st["msg_no"], now, True,
f["confidence"])
        _ev(st, "fact", field=f["field"], value=f["value"], his_words=f["his_words"],
msg_no=st["msg_no"],
                source="chat", confidence=f["confidence"])
    for sig in ("interest", "purchase_intent", "timing_objection", "prefers_call"):
        if L[sig]["on"]:
            mm.set_signal(rec, sig, now, L[sig]["evidence"])
            _ev(st, "signal", name=sig, evidence=L[sig]["evidence"])
    return st

# ----- decide
def n_decide(st):
    rec = st["rec"]
    if st["kind"] == "app_open":
        st["decision"] = Decision("GREETING", "R-GREET")
    else:
        st["decision"] = decide(st["labels"], rec, st["now"], st.get("limited", False))
    if st["decision"].action == "PAUSE_SELLING":
        rec["session"]["selling_paused"] = True
    return st

# ----- retrieve
_STOP = set("hai hain kya ka ki ke ko se me mein aur ya the tha thi ho hoon main aap mujhe mera meri is us
ye yeh wo woh "
        "the a an is are to of in on for and or i you me my what how".split())

def n_retrieve(st):
    d, rec = st["decision"], st["rec"]
    st["hits"], st["plan_row"], st["why_line"], st["past"] = [], None, "", []
    q = " ".join([st["masked"]] + st["labels"].get("topics", []))
    K = st["kb"]
    if d.action == "REFUSE_AND_TEACH":
        # always the Logic · Risk · Exit lesson first, then the closest lesson to his question
        base = K.search("logic risk exit checklist before any trade", top_k=1, categories=["Lesson"])
        more = [h for h in K.search(q, top_k=2, categories=["Lesson"]) if not base or h["chunk_id"] !=
base[0]["chunk_id"]]
        st["hits"] = (base + more)[:2]
    elif d.action in ("ANSWER_EDUCATION", "ANSWER", "ANSWER_ONLY", "LOG_GRIEVANCE", "GREETING"):
        st["hits"] = K.search(q) if q.strip() else []
    elif d.action == "ANSWER_SUPPORT":
        st["hits"] = K.search(q, categories=["FAQ"])
        if not st["hits"] or is_placeholder(st["hits"][0]):
            # no approved answer → never guess: a case goes to the team (FX-19)
            st["decision"] = Decision("SUPPORT_CASE", "R17-NOAPPROVED", fixed_line="FX-19")
    elif d.action == "ANSWER_PRICE":
        st["plan_row"], st["why_line"] = PACK.plan_for_profile(rec["facts"])
    if st["labels"]["labels"].get("refers_to_past", {}).get("on"):
        words = [w for w in re.findall(r"\w+", st["masked"].lower()) if w not in _STOP and len(w) > 2]
        for m in reversed(rec["messages"][:-1]):
            if any(w in m["text"].lower() for w in words):
                st["past"].append(f"{m['at'][:16].replace('T', ' ')} {m['role']}: {m['text'][:200]}")
            if len(st["past"]) >= 3:
                break
    return st

# ----- compose
def n_compose(st):
    rec, now, d = st["rec"], st["now"], st["decision"]
    lang = st["labels"].get("language") or rec["profile"].get("language", "hinglish")
    name = rec["profile"].get("name") or ""

```

```

st["bubbles"], st["ai_data"] = [], {}
B = st["bubbles"]
# 1. Disclosure before anything else, once in his life (FX-01)
if not rec["journey"]["disclosed"]:
    B.append({"id": "FX-01", "kind": "fixed", "text": PACK.fixed("FX-01", lang, name=name)})
    rec["journey"]["disclosed"] = True
# 2. The fixed line for this action, with values filled by code
fx = d.fixed_line
if d.action in ("HAND_OVER_PERSON",):
    fx = "FX-08" if policy.in_calling_hours(now) else "FX-07"
if fx:
    vals = {"name": name, "limit": S.get("education_questions_per_day", 30)}
    if fx == "FX-10":
        vals["case_no"] = open_case(rec, st["store"], "grievance", st["masked"], now)
        _ev(st, "case", case_no=vals["case_no"], kind="grievance", text=st["masked"])
    if fx == "FX-19":
        vals["case_no"] = open_case(rec, st["store"], "support", st["masked"], now)
        _ev(st, "case", case_no=vals["case_no"], kind="support", text=st["masked"])
    if fx in ("FX-07", "FX-08", "FX-14"):
        vals["when"] = when_phrase(now, lang)
    if fx in ("FX-07", "FX-08"):
        cb = request_callback(rec, "person", now, vals["when"])
        _ev(st, "callback", **cb)
    if fx == "FX-14":
        plan, _ = PACK.plan_for_profile(rec["facts"])
        vals["plan_line"] = (f"{plan['plan_name']} - ₹{int(float(plan['price_inr']))}, {
({plan['duration']})}. "
                                if plan else "")
        cb = request_callback(rec, "purchase", now, vals["when"])
        _ev(st, "callback", **cb)
    if fx == "FX-13":
        slots, phrases = two_slots(now, lang)
        vals["slot_1"], vals["slot_2"] = phrases
        cb = request_callback(rec, "call_preference", now, " / ".join(phrases), [iso(s) for s in slots])
        _ev(st, "callback", **cb)
    if fx == "FX-09":
        rec["journey"]["consent_line_given"] = True
    B.append({"id": fx, "kind": "fixed", "text": PACK.fixed(fx, lang, **vals)})
# 3. Tanya's own words, unless the action is fixed-only
if d.action not in FIXED_ONLY and d.action not in ("SUPPORT_CASE", "FIXED_GATE"):
    system, ex_ids = build(d.action, rec, st["labels"], mm.trial_day(rec, now), st["hits"], d.addon,
        d.addon_detail, st.get("plan_row"), st.get("why_line"))
    if st.get("past"):
        system = system.replace("TASK: REPLY", "EARLIER MESSAGES (quote with their date if he asks):\n"
                                + "\n".join(st["past"]) + "\n\nTASK: REPLY")
    msgs = mm.history_for_prompt(rec)
    if st["kind"] == "app_open":
        msgs = msgs + [{"role": "user", "content": "(he opened the app)"}]
    res = st["llm"].call("reply", d.tier, system, msgs, json_mode=True,
        temperature=S.get("temperature_reply", 0.4),
        timeout=S.get("timeout_reply_seconds", 40), max_tokens=900)
    _usage(st, res)
    st["golden_used"] = ex_ids
    reply = (res.data or {}).get("reply", "").strip() if res.ok else ""
    if reply:
        st["ai_data"] = res.data
        B.append({"id": "AI", "kind": "ai", "text": reply})
    else:
        rec["session"]["failed"] += 1
        B.append({"id": "FX-12", "kind": "fixed", "text": PACK.fixed("FX-12", lang)})
        st["notes"] = st.get("notes", []) + [f"reply call failed: {res.error}"]
# 4. Chat-first line after the first useful answer in his trial (FX-02)
if d.action in USEFUL_ANSWERS and not rec["journey"]["chat_first_given"] and
rec["profile"].get("consent"):
    B.append({"id": "FX-02", "kind": "fixed", "text": PACK.fixed("FX-02", lang)})
    rec["journey"]["chat_first_given"] = True
return st

# ----- guard
def n_guard(st):
    rec, d = st["rec"], st["decision"]
    lang = st["labels"].get("language", "hinglish")
    st["guard"] = {"net_blocked": False, "net_hits": [], "warnings": [], "ai_pass": None, "ai_problems": [],
        "replaced": False}
    ai = [b for b in st["bubbles"] if b["kind"] == "ai"]
    if ai:
        b = ai[0]
        allowed = PACK.allowed_amounts()
        for m in rec["messages"]:
            if m["role"] == "user":

```

```

        allowed |= amounts(m["text"])
    for h in st.get("hits", []):
        allowed |= amounts(h["text"])
    blocked, hits, warns = net(b["text"], allowed)
    st["guard"].update(net_blocked=blocked, net_hits=hits, warnings=warns)
    if not blocked:
        ok, problems, res = ai_check(st["llm"], b["text"], approved_text_for(st.get("hits"), d.action))
        _usage(st, res)
        st["guard"].update(ai_pass=ok, ai_problems=problems)
    if blocked or not st["guard"]["ai_pass"]:
        st["guard"]["replaced"] = True
        rec["session"]["failed"] += 1
        st["ai_data"] = {}
        b.update(id="FX-12", kind="fixed", text=PACK.fixed("FX-12", lang), blocked_text=b["text"])
    else:
        b["text"] = emoji_rule(b["text"], allowed=d.action not in NO_EMOJI)
    # the fixed disclaimer after education or a refusal, once per session (FX-17)
    if d.action in ("ANSWER_EDUCATION", "REFUSE_AND_TEACH") and not rec["session"]["disclaimer_shown"] \
        and any(b["kind"] == "ai" for b in st["bubbles"]):
        st["bubbles"].append({"id": "FX-17", "kind": "fixed", "text": PACK.fixed("FX-17", lang)})
        rec["session"]["disclaimer_shown"] = bool(S.get("disclaimer_once_per_session", True))
    return st

```

----- after

```

def n_after(st):
    rec, now, store = st["rec"], st["now"], st["store"]
    d = st.get("decision") or Decision("NONE", st.get("gate_reason", ""))
    st.setdefault("bubbles", [])
    st.setdefault("guard", {})
    st.setdefault("labels", defaults(st.get("masked", "")))
    delivered_ai = any(b["kind"] == "ai" for b in st["bubbles"])
    data = st.get("ai_data") or {}
    # counters and journey
    if d.action in COUNTS_AS_EDUCATION and delivered_ai:
        rec["counters"]["education_used"] += 1
    if data.get("asked_field") and d.addon == "ASK_MISSING_FACT":
        rec["session"]["profile_q"] += 1
    if d.addon == "SHOW_VALUE_CARD" and delivered_ai:
        rec["journey"]["cards_shown"].append(d.addon_detail["card"])
        _ev(st, "value_card", card=d.addon_detail["card"])
    if data.get("last_promise"):
        rec["journey"]["last_promise"] = data["last_promise"]
        rec["journey"]["last_promise_at"] = iso(now)
    if st["kind"] == "app_open" and st["bubbles"]:
        rec["journey"]["greeted_at"] = iso(now)
    # her messages, word for word
    for b in st["bubbles"]:
        n = mm.add_message(rec, "assistant", b["text"], now, meta={"id": b["id"], "action": d.action})
        _ev(st, "message", role="assistant", n=n, text=b["text"], line_id=b["id"], action=d.action)
    # temperature and brief (code only)
    temp, evidence = temperature(rec, now)
    rec["last_temperature"] = {"label": temp, "evidence": evidence, "at": iso(now)}
    st["brief"] = lead_brief(rec, now, temp, evidence)
    # cost ledger
    cost = round(sum(c["cost_inr"] for c in st["llm_calls"]), 4)
    today_total = store.ledger_add(cost, now) if cost else store.ledger_get(now)
    # trace (AI-C15)
    L = st["labels"]["labels"]
    st["trace"] = {
        "at": stamp(now), "user_id": st["user_id"], "kind": st["kind"], "session": rec["session"]["id"],
        "trial_day": mm.trial_day(rec, now), "gate": st.get("gate_reason"),
        "masked": st.get("masked_kinds"), "injection": st.get("injection"),
        "language": st["labels"].get("language"), "mood": st["labels"].get("mood"),
        "labels_on": {k: v["evidence"] for k, v in L.items() if v["on"]},
        "understand_failed": st["labels"].get("failed", False),
        "new_facts": st["labels"].get("new_facts", []),
        "action": d.action, "reason": d.reason, "addon": d.addon, "addon_detail": d.addon_detail,
        "tier": d.tier, "knowledge": [{"h": h["doc_id"]} for h in st.get("hits", [])],
        "golden": st.get("golden_used", []), "guard": st["guard"],
        "bubbles": [b["id"] for b in st["bubbles"]],
        "temperature": temp, "counters": {"education_used": rec["counters"]["education_used"],
                                         "education_limit": S.get("education_questions_per_day", 30),
                                         "failed_this_session": rec["session"]["failed"],
                                         "profile_q_this_session": rec["session"]["profile_q"]},
        "suppressed": mm.suppression(rec)[1], "llm_calls": st["llm_calls"], "cost_inr": cost,
        "spend_today_inr": round(today_total, 2), "spend_alert": policy.spend_alert(store, now),
        "content_version": PACK.version_string(), "notes": st.get("notes", []), "cold_start":
    }
    st.get("cold_start"),
    }
    _ev(st, "trace", trace=st["trace"])
    _ev(st, "state", version=rec["version"] + 1, temperature=temp, mode=rec["mode"]["state"],

```

```

        journey=rec["journey"], counters=rec["counters"])
    if st["kind"] == "message" or st["bubbles"]:
        _ev(st, "brief", text=st["brief"])
    store.save(rec)
    store.emit(st["events"])
    return st

```

21.4 graph.py — LangGraph wiring and the plain loop

"""The turn as a graph – LangGraph when installed (Backbone: 'LangGraph for bounded workflows'), or the same steps in a plain loop.

Plain English: the eight steps of turn.py are the 'nodes'. The only branch is after the gate: go on to understand, jump to compose for a fixed line, or stop (silent). Code decides every branch – the AI never picks the path. USE_LANGGRAPH=0 in .env forces the plain loop; both give the same result (tests check this).

```

from typing import Any, TypedDict

```

```

from . import turn
from .settings import S

```

```

ORDER = ["load", "gate", "understand", "decide", "retrieve", "compose", "guard", "after"]

```

```

NODES = {"load": turn.n_load, "gate": turn.n_gate, "understand": turn.n_understand, "decide": turn.n_decide,
        "retrieve": turn.n_retrieve, "compose": turn.n_compose, "guard": turn.n_guard, "after":

```

```

    turn.n_after}

```

```

class TurnState(TypedDict, total=False):

```

```

    """Everything one turn carries from step to step."""

```

```

    user_id: str
    kind: str
    text: str
    event_id: str
    now: Any
    store: Any
    llm: Any
    kb: Any
    seed_fn: Any
    rec: dict
    masked: str
    masked_kinds: list
    injection: bool
    msg_no: Any
    new_session: bool
    cold_start: bool
    gate: str
    gate_line: Any
    gate_reason: str
    limited: bool
    labels: dict
    decision: Any
    hits: list
    plan_row: Any
    why_line: str
    past: list
    bubbles: list
    ai_data: dict
    golden_used: list
    guard: dict
    notes: list
    llm_calls: list
    events: list
    brief: str
    trace: dict

```

```

def _langgraph_app():

```

```

    try:

```

```

        from langgraph.graph import END, START, StateGraph

```

```

    except Exception:

```

```

        return None

```

```

    g = StateGraph(TurnState)

```

```

    for name, fn in NODES.items():

```

```

        g.add_node(name, fn)

```

```

    g.add_edge(START, "load")

```

```

    g.add_edge("load", "gate")

```

```

    g.add_conditional_edges("gate", turn.route_after_gate,

```

```

        {"understand": "understand", "compose": "compose", "after": "after"})

```

```

    g.add_edge("understand", "decide")

```

```

g.add_edge("decide", "retrieve")
g.add_edge("retrieve", "compose")
g.add_edge("compose", "guard")
g.add_edge("guard", "after")
g.add_edge("after", END)
return g.compile()

_APP = None

def run_plain(state):
    """The same path as the graph, as a simple loop."""
    state = NODES["load"](state)
    state = NODES["gate"](state)
    nxt = turn.route_after_gate(state)
    if nxt == "understand":
        for n in ("understand", "decide", "retrieve", "compose", "guard"):
            state = NODES[n](state)
    elif nxt == "compose":
        for n in ("compose", "guard"):
            state = NODES[n](state)
    return NODES["after"](state)

def run_turn(state):
    """Run one turn. state needs: user_id, kind, text, now, store, llm, kb, seed_fn."""
    global _APP
    state.setdefault("llm_calls", [])
    state.setdefault("events", [])
    state.setdefault("hits", [])
    use_lg = S.env("USE_LANGGRAPH", "1") != "0"
    if use_lg:
        if _APP is None:
            _APP = _langgraph_app() or False
        if _APP:
            return _APP.invoke(state)
    return run_plain(state)

```

21.5 memory_model.py — Customer Memory rules

```

"""AI-C25 Customer Memory – the record kept for each user and its rules.

Plain English: one record per user holds his profile, the facts he or the
CRM gave (each with source, date, day, time, stated/inferred), the journey
(trial day, last promise, cards shown), the current session, daily counters,
buying signals, cases, callbacks, the chat mode (BOT/HUMAN) and his messages.

Rules implemented here (Architecture v3.2 §7):
- Which fact wins: his latest words for self-reported facts; system for account facts;
  an agent note is never changed – a different value is kept as a conflict for the agent.
- Known facts are never asked again; at most N missing ones per session.
- Her own replies are never a source of facts.
"""

from .settings import S
from .timeutil import iso, parse, stamp, day_key

PROFILE_ASK_ORDER = ["experience", "segment", "main_pain", "time_available", "goal"]
KEEP_MESSAGES = 300 # older messages live in MySQL (persister) – searched by a background job

def new_record(user_id: str, seed: dict, now) -> dict:
    """A fresh record, from the CRM (production) or a test user (dev)."""
    start = now.replace(hour=9, minute=0, second=0, microsecond=0)
    days_ago = int(seed.get("trial_start_days_ago", 0))
    from datetime import timedelta
    trial_start = seed.get("trial_start") or iso(start - timedelta(days=days_ago))
    rec = {
        "user_id": user_id,
        "conversation_id": seed.get("conversation_id", user_id),
        "profile": {"name": seed.get("name", ""), "language": seed.get("language", "hinglish"),
                    "consent": bool(seed.get("consent", False)), "trial_start": trial_start,
                    "plan": seed.get("plan", "trial"), "source": seed.get("source", "test_users")},
        "facts": {}, "conflicts": [],
        "journey": {"disclosed": False, "chat_first_given": False, "last_promise": "", "last_promise_at":
                    "cards_shown": [], "greeted_at": ""},
        "session": {}, "sessions_count": 0, "session_notes": [],
        "counters": {"day": "", "education_used": 0, "user_msgs_total": 0},
        "signals": {}, "cases": [], "callbacks": [],

```

```

        "mode": {"state": "BOT", "since": iso(now), "by": "system"},
        "messages": [], "version": 0,
    }
    for f in seed.get("facts", []):
        rec["facts"][f["field"]] = {"value": f["value"], "his_words": f.get("his_words", ""),
                                     "source": f.get("source", "crm_field"), "msg_no": None,
                                     "at": iso(now), "stated": True, "confidence": 1.0,
                                     "note": f.get("note", ""), "history": []}

    return rec

# ----- sessions and days
def ensure_session(rec, now) -> bool:
    """Start a new session after a silence longer than session_gap_minutes. Returns True if new."""
    gap = S.get("session_gap_minutes", 30)
    s = rec.get("session") or {}
    if s and (now - parse(s["last"])).total_seconds() <= gap * 60:
        s["last"] = iso(now)
        return False
    rec["sessions_count"] += 1
    rec["session"] = {"id": rec["sessions_count"], "started": iso(now), "last": iso(now),
                     "profile_q": 0, "small_talk": 0, "failed": 0, "abuse": 0, "flirt": 0,
                     "selling_paused": False, "disclaimer_shown": False, "user_msgs": 0}

    return True

def ensure_day(rec, now):
    """Daily counters reset at 00:00 IST."""
    d = day_key(now)
    if rec["counters"].get("day") != d:
        rec["counters"]["day"] = d
        rec["counters"]["education_used"] = 0

def trial_day(rec, now) -> int:
    """Day 1 = the day the trial started."""
    start = parse(rec["profile"]["trial_start"])
    return max(1, (now.date() - start.date()).days + 1)

# ----- messages
def add_message(rec, role, text, now, delivered=True, meta=None) -> int:
    n = (rec["messages"][-1]["n"] + 1) if rec["messages"] else 1
    rec["messages"].append({"n": n, "role": role, "text": text, "at": iso(now),
                           "delivered": delivered, "meta": meta or {}})
    if len(rec["messages"]) > KEEP_MESSAGES:
        rec["messages"] = rec["messages"][-KEEP_MESSAGES:]
    return n

def history_for_prompt(rec, n=None):
    """The last N delivered messages, word for word (Architecture §7.6.4)."""
    n = n or S.get("history_messages_in_prompt", 10)
    msgs = [m for m in rec["messages"] if m.get("delivered", True)]
    return [{"role": "assistant" if m["role"] == "assistant" else "user", "content": m["text"]}
            for m in msgs[-n:]]

# ----- facts
SYSTEM_SOURCES = {"crm_field", "agent_note", "app"}

def set_fact(rec, field, value, his_words, source, msg_no, now, stated=True, confidence=0.9):
    """Store a fact with its source. Applies 'which fact wins' and records conflicts."""
    old = rec["facts"].get(field)
    entry = {"value": value, "his_words": his_words, "source": source, "msg_no": msg_no,
             "at": iso(now), "stated": stated, "confidence": confidence, "note": "", "history": []}
    if old:
        entry["history"] = (old.get("history", []) + [{"k": old[k] for k in ("value", "source", "at")}])[-5:]
        if old["source"] in SYSTEM_SOURCES and source == "chat" and \
            old["value"].strip().lower() != str(value).strip().lower():
            # The agent's note is never changed; the difference is kept for the agent (§9.4)
            rec["conflicts"].append({"field": field, "agent_value": old["value"], "agent_source":
                                     old["source"],
                                     "customer_value": value, "his_words": his_words, "at": iso(now),
                                     "status": "open"})
        rec["facts"][field] = entry # his latest words are used in conversation (§7.2)

def known_fields(rec) -> set:
    return {k for k, v in rec["facts"].items() if v.get("value")}

```

```

def missing_profile_fields(rec) -> list:
    known = known_fields(rec)
    return [f for f in PROFILE_ASK_ORDER if f not in known]

def fact_lines(rec) -> list:
    """Known facts for the prompt – 'never ask these again'."""
    out = []
    for k, v in rec["facts"].items():
        who = {"chat": "he told Tanya", "agent_note": "agent note", "crm_field": "CRM", "app":
"app"}.get(v["source"], v["source"])
        words = f' – his words: "{v["his_words"]}"' if v.get("his_words") else ""
        out.append(f"{k}: {v['value']} ({who}, {stamp(parse(v['at']))}){words}")
    return out

def pain_text(rec) -> str:
    """His pain in his own words, for value cards and price answers."""
    parts = []
    for k in ("main_pain", "past_loss"):
        f = rec["facts"].get(k)
        if f:
            parts.append(f.get("his_words") or f["value"])
    return " | ".join(parts)

# ----- signals
def set_signal(rec, name, now, evidence=""):
    rec["signals"][name] = {"at": iso(now), "evidence": evidence}

def signal_active(rec, name, now, hours=None) -> bool:
    s = rec["signals"].get(name)
    if not s:
        return False
    if hours is None:
        return True
    return (now - parse(s["at"])).total_seconds() <= hours * 3600

# ----- suppression
def suppression(rec):
    """Selling is blocked by distress this session, an open grievance, or no consent (§8.5)."""
    reasons = []
    if rec["session"].get("selling_paused"):
        reasons.append("distress this session")
    if any(c["kind"] == "grievance" and c["status"] == "open" for c in rec["cases"]):
        reasons.append("open grievance")
    if not rec["profile"].get("consent"):
        reasons.append("no consent")
    return bool(reasons), reasons

```

21.6 scoring.py — Lead temperature

```

"""AI-C06 Lead Scoring + AI-C07 Trial Intelligence – lead temperature (Architecture v3.2 §8).

Plain English: curiosity is not a buying decision. Interest, purchase intent and
readiness are kept separate, each with evidence and an expiry, and CODE combines
them into Hot / Warm / Nurture / Not ready. Suppression (distress, open grievance,
no consent) always wins – never Hot, whatever words appear.
These are qualification rules, not a probability of buying.
"""

from . import memory_model as mm
from .settings import S

def readiness(rec, now):
    """Profile fits (experience and segment known), no open timing objection, no refused call."""
    known = mm.known_fields(rec)
    reasons = []
    if not {"experience", "segment"} <= known:
        reasons.append("experience/segment not known")
    if mm.signal_active(rec, "timing_objection", now, 7 * 24):
        reasons.append("timing objection open")
    if mm.signal_active(rec, "refused_call", now):
        reasons.append("refused a call")
    return not reasons, reasons

```

```

def temperature(rec, now):
    """Returns (label, evidence_lines)."""
    suppressed, why = mm.suppression(rec)
    intent = mm.signal_active(rec, "purchase_intent", now, S.get("intent_expiry_hours", 48))
    interest = mm.signal_active(rec, "interest", now, S.get("interest_expiry_hours", 72))
    ready, not_ready = readiness(rec, now)
    ev = []
    for name in ("purchase_intent", "interest", "timing_objection", "prefers_call"):
        s = rec["signals"].get(name)
        if s:
            ev.append(f"{name}: \"{s.get('evidence', '')}\" ({s['at'][:16].replace('T', ' ')}")
    if suppressed:
        ev.append("suppressed: " + " ".join(why))
    if intent and ready and not suppressed:
        return "Hot", ev
    known = mm.known_fields(rec)
    if interest and not suppressed and "experience" in known and ("main_pain" in known or "past_loss" in
known):
        return "Warm", ev
    if rec["counters"].get("user_msgs_total", 0) >= 3 and not intent:
        if not ready:
            ev.append("not ready: " + " ".join(not_ready))
        return "Nurture", ev
    return "Not ready", ev

```

21.7 handoff.py — Hand-offs, cases, callbacks, honest times

```

"""AI-C18 Human Approval Queue – cases, callbacks and honest times.

Plain English:
- Grievances and unanswerable support questions become a case with a number.
- Hand-overs become callbacks with three states: requested → booked → completed.
- Tanya only promises a time inside the confirmed calling window; at night she
  names the next opening time, never an invented one (Architecture §12).
"""

from datetime import timedelta

from .policy import in_calling_hours
from .settings import S
from .timeutil import iso, hm

_DAY_WORDS = {
    "hinglish": ("aaj", "kal"),
    "english": ("today", "tomorrow"),
    "hindi": ("आज", "कल"),
}

def _clock(h, m, lang):
    """10:00 → '10 baje' / '10:00 AM' / '10 बजे'."""
    if lang == "english":
        hh = h % 12 or 12
        return f"{hh}:{m:02d} {'AM' if h < 12 else 'PM'}"
    part = "subah" if h < 12 else ("dopahar" if h < 16 else "shaam" if h < 19 else "raat")
    part_hi = {"subah": "सुबह", "dopahar": "दोपहर", "shaam": "शाम", "raat": "रात"}[part]
    hh = h % 12 or 12
    mins = f"{m:02d}" if m else ""
    if lang == "hindi":
        return f"{part_hi} {hh}{mins} बजे"
    return f"{part} {hh}{mins} baje"

def phrase_at(dt, now, lang):
    """'kal subah 10 baje' / 'tomorrow at 10:00 AM'."""
    today, tomorrow = _DAY_WORDS.get(lang, _DAY_WORDS["hinglish"])
    if dt.date() == now.date():
        d = today
    elif dt.date() == (now + timedelta(days=1)).date():
        d = tomorrow
    else:
        d = dt.strftime("%a %d-%b")
    if lang == "english":
        return f"{d} at {_clock(dt.hour, dt.minute, lang)}"
    return f"{d} {_clock(dt.hour, dt.minute, lang)}"

def next_opening(now):
    """The next time the calling window opens (today or tomorrow)."""
    sh, sm = hm(S.get("calling_window_start", "10:00"))
    opening = now.replace(hour=sh, minute=sm, second=0, microsecond=0)

```

```

    if now >= opening:
        opening += timedelta(days=1)
    return opening

def when_phrase(now, lang):
    """What Tanya may promise for a person's contact time."""
    if in_calling_hours(now):
        return {"hinglish": "jaladi hi", "english": "shortly", "hindi": "जल्द ही"}.get(lang, "jaladi hi")
    return phrase_at(next_opening(now), now, lang)

def two_slots(now, lang):
    """Two call slots, whole hours inside the calling window, at least 1 hour from now."""
    sh, _ = hm(S.get("calling_window_start", "10:00"))
    eh, _ = hm(S.get("calling_window_end", "19:00"))
    slots = []
    day = now
    while len(slots) < 2:
        for h in range(sh + 1, eh):
            cand = day.replace(hour=h, minute=0, second=0, microsecond=0)
            if cand >= now + timedelta(hours=1) and len(slots) < 2:
                if not slots or (cand - slots[-1]).total_seconds() >= 3 * 3600:
                    slots.append(cand)
            day = (day + timedelta(days=1)).replace(hour=0, minute=0)
    return slots, [phrase_at(s, now, lang) for s in slots]

def open_case(rec, store, kind, text, now) -> str:
    case_no = store.next_case_no(now)
    rec["cases"].append({"case_no": case_no, "kind": kind, "status": "open", "at": iso(now), "text":
text[:300]})
    return case_no

def request_callback(rec, kind, now, when_text, slot=None) -> dict:
    """kind: person | purchase | call_preference. State starts at 'requested'."""
    for cb in rec["callbacks"]:
        if cb["kind"] == kind and cb["state"] in ("requested", "booked"):
            return cb # one open callback per kind – no duplicates
    cb = {"id": f"CB-{rec['user_id']}-{len(rec['callbacks']) + 1}", "kind": kind, "state": "requested",
        "requested_at": iso(now), "when_text": when_text, "slot": slot}
    rec["callbacks"].append(cb)
    return cb

```

21.8 guard_input.py — Masking and injection check

```

"""AI-C23 Prompt Injection Detector + input masking (part of AI-C01 Gateway duties).

Plain English: before any AI sees a message, private numbers are hidden
(phone, email, PAN, card, Aadhaar, bank account, OTP) and obvious attempts
to trick the AI ("ignore your instructions") are caught.
Nothing here calls an AI – it is plain rules, so it is fast and testable.
"""

import re

# Order matters: longer / more specific patterns first.
_MASKS = [
    ("EMAIL", re.compile(r"[\w.-]+@[ \w.-]+\.[\w.-]+")),
    ("PAN", re.compile(r"\b[A-Z]{5}[0-9]{4}[A-Z]\b", re.I)),
    ("CARD", re.compile(r"\b(?:\d[ -]?)\{13,19\}\b")),
    ("AADHAAR", re.compile(r"\b\d{4}[ -]\d{4}[ -]\d{4}\b")),
    ("PHONE", re.compile(r"?:\+91[ -]?\b0?\b[6-9]\d{9}\b")),
    ("OTP", re.compile(r"\b(?:otp|one[ -]time[ -]password)\D{0,12}(\d{4,8})\b", re.I)),
    ("BANK", re.compile(r"\b\d{9,18}\b")),
]

_INJECTION = [
    r"ignore (all|your|the|previous|above) [^\.]{0,30} (instruction|rule|prompt)",
    r"(system|developer) prompt",
    r"you are now",
    r"act as (a|an|my)",
    r"pretend (to be|you are)",
    r"forget (your|all|the) (rules|instructions)",
    r"jailbreak",
    r"developer mode",
    r"(pichle|purane) (instructions|rules) (bhool|ignore)",
    r"apne rules (bhool|ignore|todo)",
]

_INJ = re.compile("|".join(_INJECTION), re.I)

```

```
def mask(text: str):
    """Return (masked_text, list_of_kinds_masked)."""
    found = []
    out = text
    for kind, rx in _MASKS:
        if kind == "OTP":
            def _otp(m):
                return m.group(0).replace(m.group(1), "[OTP]")
            new = rx.sub(_otp, out)
        else:
            new = rx.sub(f"[{kind}]", out)
        if new != out:
            found.append(kind)
        out = new
    return out, found

def injection(text: str) -> bool:
    """True when the message tries to change the AI's instructions."""
    return bool(_INJ.search(text or ""))
```

21.9 guard_output.py — SEBI net, ₹ check, emoji rule, AI check

```
"""AI-C14 Guardrail & Compliance – SEBI guard layers 2 and 3 on Tanya's written reply.

Plain English:
- Layer 1 (route before writing) is in decider.py: trade questions go to REFUSE_AND_TEACH.
- Layer 2 (this file, ai_check): a separate small AI call checks her reply against a strict list.
- Layer 3 (this file, net): plain rules – forbidden words and patterns, every ₹ figure must
  be an approved price or a figure he said himself, the emoji rule, and the fixed disclaimer.
Fixed lines are pre-approved and are not checked again.
"""

import json
import re

from .content_pack import PACK
from .settings import S

# ----- ₹ figures
_AMT = [
    re.compile(r"(?:₹|\brs\.|?|\binr)\s*([d,]+(?:\.\d+)?)\s*(lakh|lac|crore|cr|k|hazaar|hazar|thousand)?\b",
re.I),
    re.compile(r"\b([d,]+(?:\.\d+)?)\s*(lakh|lac|crore|cr|hazaar|hazar|thousand)?\s*(?:rupaye|rupees|rupay|rupiya|रुपये)", re.I),
    re.compile(r"\b([d,]+(?:\.\d+)?)\s*(lakh|lac|crore)\b", re.I),
]
_MULT = {"lakh": 100_000, "lac": 100_000, "crore": 10_000_000, "cr": 10_000_000,
        "k": 1_000, "hazaar": 1_000, "hazar": 1_000, "thousand": 1_000}

def amounts(text: str) -> set:
    """Every rupee amount mentioned, as whole rupees. '₹3 lakh' → 300000, '₹15,000' → 15000."""
    out = set()
    for rx in _AMT:
        for m in rx.finditer(text or ""):
            num = m.group(1).replace(",", "")
            try:
                v = float(num)
            except ValueError:
                continue
            unit = (m.group(2) or "").lower() if m.lastindex and m.lastindex >= 2 else ""
            v *= _MULT.get(unit, 1)
            out.add(int(round(v)))
    return out

# ----- emoji rule
_EMOJI = re.compile("[\U0001F000-\U0001FAFF*-\U0001F600-\U0001F64F]")

def emoji_rule(text: str, allowed: bool) -> str:
    """At most one emoji; none when not allowed (distress, grievance, refusal, money, hand-off)."""
    found = [m for m in _EMOJI.finditer(text) if m.group(0) not in ("", "")]
    if not found:
        return text
    keep_first = allowed
    out, seen = [], False
    for ch in text:
```

```

        if _EMOJI.match(ch):
            if ch in (" ", "\n"):
                if keep_first and seen and out and _EMOJI.match(out[-1]):
                    out.append(ch)
                    continue
                if keep_first and not seen:
                    out.append(ch)
                    seen = True
                continue
            out.append(ch)
        return re.sub(r"[ \t]{2,}", " ", "".join(out)).strip()

# ----- layer 3 net
def _sentences(text):
    return [s for s in re.split(r"(?<=[!?!])\s+|\n", text or "") if s.strip()]

def net(reply: str, allowed_amounts: set):
    """Returns (blocked: bool, hits: list, warnings: list)."""
    lists = PACK.guard
    hits, warns = [], []
    denials = [d.lower() for d in lists.get("denial_words", [])]
    for sent in _sentences(reply):
        low = sent.lower()
        for p in lists.get("promise", []):
            if re.search(p["pattern"], low, re.I):
                if not any(d in low for d in denials):
                    hits.append(f"{p['id']} {p['why']}: \"{sent.strip()[:80]}\"")
        for p in lists.get("hard", []):
            if re.search(p["pattern"], low, re.I):
                hits.append(f"{p['id']} {p['why']}: \"{sent.strip()[:80]}\"")
        for p in lists.get("warn", []):
            if re.search(p["pattern"], low, re.I):
                warns.append(f"{p['id']} {p['why']}")
    bad_amounts = sorted(a for a in amounts(reply) if a not in allowed_amounts)
    if bad_amounts:
        hits.append("AMT ₹ figure not in pricing or his words: " + ", ".join(f"₹{a:,.}" for a in
bad_amounts))
    return bool(hits), hits, warns

# ----- layer 2 AI check
CHECK_SYSTEM = """TASK: CHECK
You are a strict SEBI compliance checker for replies written by an AI assistant of a SEBI-registered
Research Analyst.
FAIL the reply if it contains ANY of:
1. A view or prediction on market direction (up/down) for any index, stock or instrument.
2. An entry, exit, target, stop-loss level or strike for a specific instrument, or a named stock/index to
buy or sell.
3. A promise or implication of profit, returns, income, accuracy, success rate, or loss recovery (a clear
denial is fine).
4. A price, discount, plan content or benefit that is NOT in the APPROVED TEXT given below.
5. An invented human life for the assistant (eating, drinking, travelling, family, age, tiredness).
6. Addressing the user as 'tum' or 'tu'.
7. Superlatives about the company ('best', 'No. 1', 'top', 'leading') or fake urgency.
Teaching general concepts (what a stop-loss is, how to read an alert, risk management) is ALLOWED.
Explaining that there is no guarantee is ALLOWED.
Return ONLY JSON: {"pass": true|false, "problems": ["short reason", ...]}"""

def ai_check(llm, reply: str, approved_text: str):
    """Layer 2. Returns (passed, problems, LLMResult). A failed check call = not passed (safe side)."""
    msg = f"APPROVED TEXT (prices, plans, knowledge the reply may use):\n{approved_text or
'(none)}'\n\nREPLY TO CHECK:\n{reply}"
    res = llm.call("check", "fast", CHECK_SYSTEM, [{"role": "user", "content": msg}], json_mode=True,
        temperature=S.get("temperature_check", 0.0), timeout=S.get("timeout_check_seconds", 20),
        max_tokens=400)
    if not res.ok:
        return False, [f"check call failed: {res.error}"], res
    passed = bool(res.data.get("pass"))
    return passed, [str(p) for p in res.data.get("problems", [])][:5], res

def approved_text_for(hits, action):
    """What the checker may accept as approved: knowledge used + pricing (for price answers)."""
    parts = [h["text"] for h in hits or []]
    parts.append(json.dumps([k: r[k] for k in ("plan_name", "duration", "price_inr", "inclusions")
        for r in PACK.pricing], ensure_ascii=False))
    return "\n".join(parts)

```

21.10 knowledge.py — RAG — knowledge search

```
"""AI-C05 Knowledge / RAG – approved knowledge only.

Plain English – how RAG works here:
1. Approved files (content/kb/*.md: FAQs, lessons) are split into short pieces ('chunks').
2. For each question, the closest pieces are found by
  (a) keyword search (BM25 – always on, needs no key), and
  (b) meaning search with vectors (embeddings), when EMBEDDING_PROVIDER is set in .env.
  Both scores are combined ('hybrid').
3. The top pieces go into Tanya's prompt; she answers only from them.

Where the vectors live:
- Pilot: in this program's memory (the 'vector index' is the list below), vectors cached
  in data/kb_vectors.json so they are computed once per approved version.
- Production: the same chunks are written to MySQL orch_kb_chunks (chunks_for_db) and,
  later, to the Backbone's vector DB (Qdrant or MongoDB Vector Search – decision open).
Customer memory is NOT here – it is looked up by user ID (memory_store.py).
"""

import hashlib
import json
import math
import re
from collections import Counter

from .content_pack import PACK, parse_front_matter
from .llm import embed
from .settings import ROOT, S

# Hinglish words mapped to the English words used in the approved files
SYNONYMS = {
    "nuksaan": "loss", "nuksan": "loss", "ghata": "loss", "sl": "stop-loss stop loss", "stoploss": "stop-
loss stop loss",
    "kitna": "how much", "paisa": "capital money", "paise": "capital money", "shuru": "start",
    "samajh": "understand", "seekhna": "learn", "chart": "chart", "alert": "alert research",
    "call": "call alert", "tips": "tip call", "trade": "trade", "risk": "risk", "exit": "exit",
    "refund": "refund cancellation", "install": "install app", "notification": "notification alert",
    "locked": "locked access", "access": "access locked", "guarantee": "guarantee sure-shot",
    "pakka": "sure-shot guarantee", "accuracy": "accuracy sure-shot", "overtrading": "overtrading",
    "ce": "call option ce", "pe": "put option pe", "premium": "premium option", "expiry": "expiry option",
    "registration": "sebi registration", "sebi": "sebi registration", "fraud": "sebi registration verify",
}

_WORD = re.compile(r"[\w\u0900-\u0940\u0940-\u0980\u0980-\u09c0\u09c0-\u09ff]+", re.UNICODE)

_IGNORE = {"tanya", "ms", "tg", "level", "the", "a", "an", "is", "to", "of", "and", "or", "hai", "ka", "ki",
"ke", "kya"}

def tokens(text: str):
    out = []
    for w in _WORD.findall((text or "").lower()):
        if w in _IGNORE:
            continue
        out.append(w)
        if w in SYNONYMS:
            out.extend(SYNONYMS[w].split())
    return out

class KnowledgeIndex:
    def __init__(self):
        self.chunks = []
        self.cache_path = ROOT / "data" / "kb_vectors.json"
        self._build()

    def _build(self):
        for path, text in PACK.kb_files():
            head, body = parse_front_matter(text)
            qs = re.search(r"Question forms:\s*(.*?)\n\s*\n", body, flags=re.S)
            ans = re.search(r"Answer:\s*(.*?)\n\s*\nNever say:|Z)", body, flags=re.S)
            never = re.search(r"Never say:\s*(.*?)$", body, flags=re.S)
            answer = (ans.group(1) if ans else body).strip()
            doc = {"doc_id": head.get("doc_id", path.stem), "title": head.get("title", path.stem),
                  "category": head.get("category", ""), "status": head.get("status", ""),
                  "version": head.get("version", ""), "questions": (qs.group(1).strip() if qs else ""),
                  "never_say": (never.group(1).strip() if never else "")}
            for i, part in enumerate(self._split(answer)):
                c = dict(doc, chunk_id=f"{doc['doc_id']}#{i:02d}", text=part)
                c["search_text"] = f"{doc['title']}\n{doc['questions']}\n{part}"
```

```

        c["tokens"] = tokens(c["search_text"])
        self.chunks.append(c)
    self._bm25_prepare()
    self._vectors()

@staticmethod
def _split(answer, max_chars=900):
    """Split a long answer into pieces at blank lines; small answers stay whole."""
    if len(answer) <= max_chars:
        return [answer]
    parts, cur = [], ""
    for para in re.split(r"\n\s*\n", answer):
        if len(cur) + len(para) > max_chars and cur:
            parts.append(cur.strip())
            cur = ""
        cur += para + "\n\n"
    if cur.strip():
        parts.append(cur.strip())
    return parts

# ----- keyword search (BM25)
def _bm25_prepare(self):
    self.N = len(self.chunks)
    self.avgdl = sum(len(c["tokens"]) for c in self.chunks) / max(1, self.N)
    df = Counter()
    for c in self.chunks:
        df.update(set(c["tokens"]))
    self.idf = {w: math.log(1 + (self.N - n + 0.5) / (n + 0.5)) for w, n in df.items()}
    for c in self.chunks:
        c["tf"] = Counter(c["tokens"])

def _bm25(self, q_tokens, c, k1=1.5, b=0.75):
    s = 0.0
    dl = len(c["tokens"])
    for w in set(q_tokens):
        if w not in c["tf"]:
            continue
        tf = c["tf"][w]
        s += self.idf.get(w, 0) * tf * (k1 + 1) / (tf + k1 * (1 - b + b * dl / self.avgdl))
    return s

# ----- meaning search (vectors)
def _vectors(self):
    self.vec_model = S.env("EMBEDDING_PROVIDER", "")
    self.has_vectors = False
    if not self.vec_model:
        return
    cache = json.loads(self.cache_path.read_text()) if self.cache_path.exists() else {}
    need = [c for c in self.chunks if self._key(c) not in cache]
    if need:
        vecs = embed([c["search_text"] for c in need])
        if vecs is None:
            return
        for c, v in zip(need, vecs):
            cache[self._key(c)] = v
        self.cache_path.parent.mkdir(exist_ok=True)
        self.cache_path.write_text(json.dumps(cache))
    for c in self.chunks:
        c["vec"] = cache.get(self._key(c))
    self.has_vectors = all(c.get("vec") for c in self.chunks)

def _key(self, c):
    return self.vec_model + ":" + hashlib.sha1(c["search_text"].encode()).hexdigest()

@staticmethod
def _cos(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(y * y for y in b))
    return dot / (na * nb) if na and nb else 0.0

# ----- the search
def search(self, query: str, top_k=None, categories=None):
    """Hybrid search. Returns the best chunks with scores (0..1)."""
    top_k = top_k or S.get("knowledge_top_k", 3)
    q = tokens(query)
    candids = [c for c in self.chunks if not categories or c["category"] in categories]
    if not candids:
        return []
    kw = [self._bm25(q, c) for c in candids]
    mx = max(kw) or 1.0
    scores = [k / mx for k in kw]

```

```

        if self.has_vectors:
            qv = embed([query])
            if qv:
                scores = [0.5 * s + 0.5 * max(0.0, self._cos(qv[0], c["vec"])) for s, c in zip(scores,
cands)]

        ranked = sorted(zip(scores, kw, cands), key=lambda t: -t[0])
        out = []
        for s, raw, c in ranked[:top_k]:
            if raw <= 0 and not self.has_vectors:
                continue
            out.append({k: c[k] for k in ("chunk_id", "doc_id", "title", "category", "status", "version",
"text", "never_say")} | {"score": round(s, 3)})

        return out

    def chunks_for_db(self):
        """Rows for MySQL orch_kb_chunks (production sync job)."""
        return [{"chunk_id": c["chunk_id"], "doc_id": c["doc_id"], "title": c["title"], "category":
c["category"],
                "status": c["status"], "version": c["version"], "text": c["text"],
                "content_hash": hashlib.sh1(c["search_text"].encode()).hexdigest()} for c in self.chunks]

    def is_placeholder(hit) -> bool:
        return "PLACEHOLDER" in (hit.get("status") or "").upper() or hit.get("text", "").startswith("
[PLACEHOLDER]")

```

22. Change register

1. **v1.0 — Tue 29-Sep-2026** — first issue. Replaces the demo approach with a code frame for the juniors (your direction, 29-Sep): module map with Backbone components; the eight-step turn with LangGraph; labels; decider table R00–R20 with add-ons; Customer Memory record, rules and Redis keys; RAG with hybrid keyword and vector search; three-layer SEBI guard; temperature, Lead Brief and Agent Short-Call Card; honest hand-off times; fixed lines FX-01–FX-19; reliability; 16 orch_ tables with the event map; provider adapter (no LangChain); content files; configuration table with owners; 36 rule tests passing; what is left with estimate; open points register; the rule modules printed verbatim.